Benchmarking Quantized Small Language Models for Edge-Based Structured Information Extraction: An Empirical Systems Study

Aryaman Sinha
Department of Computer Science
Vellore Institute of Technology
Vellore, India
aryaman.sinha.lko@gmail.com

Nithin Rajan
Department of Computer Science
Vellore Institute of Technology
Vellore, India
nithinrajan48@gmail.com

Johith Babu S
Department of Computer Science
Vellore Institute of Technology
Vellore, India
johithbabu2006@gmail.com

Ajitesh Mohan
Department of Computer Science
Vellore Institute of Technology
Vellore, India
ajiteshmn@gmail.com

***Abstract*—Using a Small Language Model (SLM), the data can be extracted at low-latencies while keeping the extraction private due to not sharing any PII with a centralised infrastructure. SLMs deployed on edge hardware must then be quantized to integers after training to meet strict memory constraints. Most standard benchmarks focus on measuring performance with regards to perplexity when using a language model. However, little work has been done to understand how low-bit quantization affects the performance of the structured, deterministic extraction task. Here we provide an empirical benchmark which evaluates the performance of Meta's Llama-3-8B model in 4-bit (`Q4_K_M`) and 8-bit (`Q8_0`) quantization on a consumer edge workstation device (Intel Core i9-13900HX, NVIDIA RTX 4060 Laptop GPU, 16 GB DDR5 RAM). This benchmark extracts multiple fields from academic records that are given within unstructured text with large numerical distractions, informal subject names and missing optional fields.**

We find that using 4-bit integers reduces the average inference time by 71.3% (from 3.104 ± 0.266 s to 0.892 ± 0.149 s; Welch's $t = -14.40, p = 2.46 \times 10^{-30}$). This is also reflected in the reduction of the tail latency ($P95$), which decreases by 74.2%. Our results also show that we can achieve 100.0% syntactically correct JSON. However, we find a consistent "syntax-semantic issue". Whilst the JSON is almost completely syntactically correct, both quantization methods only extract F_1 of the information correctly (a score of 0.510 ± 0.020) and have an exact match rate of 2.0%. This is largely due to issues with autoregressive attention causing the model to get confused between the target and numerical values within the JSON (room numbers and credit hours). We also see that there is a 1.0% increase in the OOM process termination rate when using 8-bit quantization. This causes major issues with the virtual memory swapping when running on 16 GB of hardware. However, this issue does not occur when using 4-bit quantization as it has a 0.0% failure rate. We create an error classification system, analyse the impact on the amount of memory required for inference and give guidance on how to create a reliable information extraction pipeline.

***Index Terms*—Edge Computing, SLMs, Quantization, Structured Information Extraction, Constrained Decoding, JSON Schema, Systems Benchmarking, Distractor Robustness.**

I. Introduction

Generative Pre-trained Transformers (GPTs) have been incorporated into production back-end systems which have revolutionized automate document processing and entity extraction and chatbots [1]. However, large language models (LLMs) such as GPT-4 and Claude 3.5 Sonnet hosted on the cloud suffer from significant issues when using them for enterprise, health care and education. Firstly, they have large privacy concerns as the candidate input needs to be sent over the wide area network. They will also cause large issues with regulation as they are covered by GDPR [2], HIPAA and FERPA. Secondly, there is a cost associated with calling the token based API multiple times. This can also cause delays with the wide area network.

Modern architectures are moving towards a solution called *edge-native computing* [3], [4]. This solution makes use of small language models (SLMs) with between 1 and 8 billion parameters and runs them locally on client machines, point of care devices and gateways. This ensures we have full control of our data, no latency as we aren't using the network and predictable operating expenditure.

Modern transformer architectures are highly memory intensive and have significant limitations on consumer hardware. An 8-Billion parameter model which is stored at 16-bit floating point precision will require 16 GB of VRAM just to store its weight matrix (FP16). This does not even take into account the extra memory needed by KV caching, activation tensors, and OS overhead [5], [6]. Most consumer laptops and edge nodes will only have 8 GB of VRAM and 16 GB of DDR5 RAM available meaning it would be impossible to run unquantized models.

Post-Training Quantization (PTQ) enables a solution to this problem by reducing the weight matrices from high-precision

floating points to low-bit integers (such as INT8/INT4) [7], [8]. This reduces the memory bandwidth requirements and allows us to reduce the weight memory requirement to 4.8-8.5 GB so we can run them locally.

When extracting information in production data pipelines we cannot allow the information extraction models to produce free text. The extracted entities need to be deterministically converted to a strict valid format like JavaScript Object Notation (JSON) [9] (RFC 8259). If any of the curly braces or quotes are not properly escaped the JSON will fail to deserialize. This means that if we have a missing curly brace, an unescaped quote, or have some unexpected text at the start of the JSON (e.g. *"Sure, here is your JSON:"*), then the JSON will fail to deserialize in our automated microservices.

Despite the rapid proliferation of quantized SLMs, existing benchmarking methodologies exhibit three critical shortcomings:

1) **Dominance of Perplexity and Prose Over Structured Extraction:** Commonly used benchmarks like MMLU [10] and GSM8K [11] mostly measure how good a model is at predicting the next token, generating grammatical sentences, or performing multiple choice reasoning on unstructured texts. They do not however measure how well a model can deterministically retain relations between unstructured entities and structured key value schemes.
2) **Equating Syntactic Validity with Semantic Truth:**Structured generation is often evaluated by simply checking if the output can be parsed using standard libraries (e.g., Python's `json.loads()`) [12], [13]. However, this gives us a false sense of precision where models may easily produce syntactically correct JSON, but the numbers in the JSON may be complete nonsense, or there may be hallucinated fields, or there may be numerical distractors.
3) **Systematic Concealment of Hardware Crashes and Telemetry:** Benchmarks often exclude failed tests, OOM exceptions and page faults at the system level to give a clean latency result. If an OOM exception isn't handled on the edge it will cause unreliability in your pipeline..

To address these fundamental issues, we conduct a comprehensive benchmark of quantized SLMs at answering system questions within a consumer device setting. We study Meta's Llama-3-8B model at Q4 (Q4_K_M) and Q8 (Q8_0) quantization levels. Our benchmark consists of 1200 synthetic academic forms with realistic noise such as numbers, synonyms and missing fields.

A. Key Contributions

The core scientific and engineering contributions of this work are summarized as follows:

- **Empirical Systems Benchmarking Framework:** We provide an open source benchmarking framework that provides un-sanitized measurement of multi-dimensional extraction accuracy (syntax, schema, field-level F_1, exact), as well as low-level hardware metrics (latency, tail $P95$, memory RSS and process faults).
- **Discovery of the Syntax-Semantics Discrepancy:** We show that a 100.0% JSON syntactic accuracy does not translate into high extraction accuracy. We see that both the INT4 and INT8 models have an extraction F_1 score of only 0.510 and an exact match rate of 2.0%. This means that the lower bit quantization methods do well at keeping the token-level syntactic features but are unable to overcome the numerical distractor issue. .
- **Characterization of the 16 GB Memory Execution Cliff:** We provide concrete evidence of real-world hardware limitations. On a 16 GB host workstation we observe a 1.0% OOM crash rate under 8-bit quantization due to high virtual memory swapping. Under 4-bit quantization the host sees zero thrashing (0.0%) and a 71.3% latency reduction ($p = 2.46 \times 10^{-30}$).
- **Formal Error Taxonomy and Edge Design Principles:** We formulate a four-tier categorical taxonomy of structured extraction failures and present actionable architectural guidelines for deploying robust SLMs in resource-bounded edge pipelines.

The remainder of this paper is structured as follows. A review of relevant work around quantization, constrained decoding and edge benchmarks is provided in Section II. Section III describes the problem setup, quantization process and algorithm used. Section IV outlines the experimentation setup, benchmark dataset and metrics. Section V discusses results including error types, and hardware data. Section VI presents best practice advice for working with models on the edge, and threats to validity. Lastly, we conclude in Section VIII.

II. Related Work

A. Post-Training Quantization of Transformers

Post-Training Quantization (PTQ) methods reduce the precision of parameters of pre-trained transformers. `LLM.int8()` was first presented by Dettmers et al. [5]. By separating out the dimensions of emergent outliers in the float representations, it was shown that there would be no catastrophic loss in performance during 8-bit matrix multiplication. Then, Frantar et al. [6] presented GPTQ, an approximate second order algorithm based on Optimal Brain Surgeon. This allowed for accurate 4-bit and 3-bit weight-only quantization. Finally, Lin et al. [7] proposed AWQ (Activation-aware Weight Quantization). It was shown that by protecting important weights which correlate with high magnitude activations can greatly reduce the loss in perplexity during aggressive 4-bit compression.

Tensor-based algorithms have been complimented by system-based solutions such as llama.cpp [14]. This introduced block quantization schemes (k-quants). Schemes include `Q4_K_M` and `Q8_0`). These are achieved by dividing the weight tensors into blocks. Non-linear scaling factors are applied. The quantized tensors are compatible with modern SIMD architectures (AVX-512,ARM Neon) and CUDA tensor cores. While there is extensive validation of the models' perplexity on common

corpora (WikiText-2,C4), the effect on deterministic entity extraction with strict JSON schemas has not been explored.

B. Constrained Decoding and Schema-Guided Generation

A number of different methods have been developed to induce autoregressive language models to generate valid structured data. Willard and Louf [12] developed the Outlines package to convert Context-Free Grammars (CFGs) and regular expressions to deterministic finite state automata (DFA). During decoding, invalid tokens are masked using the DFA to mathematically ensure that the token sequence generated adheres to the formal schema. Likewise, Zheng et al. [13] developed the SGLang package which uses RadixAttention and fast grammatical constrains to improve performance of structured programs. Finally, in the open source edge community GBNF (GGML Backus-Naur Form) grammars [14] are used to impose JSON semantics during the sampling process in `llama.cpp`.

Whilst constrained decoding does offer 100% adherence to the syntax ($V_{\text{json}} = 1.0$), there is a new issue that occurs. If the model incorrectly associates an entity or picks an incorrect numerical distractor, it will be forced by the grammar engine to produce valid tokens. This means the token sequence will be syntactically correct but semantically incorrect.

C. Structured Information Extraction Benchmarks

Information Extraction (IE) has transitioned from the traditional rule-based and Named Entity Recognition (NER) tagger based systems (e.g., CoNLL-2003 [18], OntoNotes 5.0) to generative end to end information extractors. Modern benchmarking efforts have focused around document based benchmarks such as DocVQA [19] and Few-NERD [20] that seek to extract information from unstructured layouts. Recently, benchmarks have been proposed that seek to measure Large Language Models (LLMs) ability to generate structured API outputs such as Schema-Guided Dialogue (SGD) [21] and JSON-Bench.

Existing datasets commonly benchmark models using high capacity FP16 models in a cloud setting. However, there has been little investigation into how integer quantization noise, distractor tokens within the context window and nested key-value mapping will interact on edge tier hardware.

D. Edge Systems Telemetry and Reliability

The edge computing environment is very different from a data center deployment. There is no high bandwidth memory (HBM) available on consumer grade devices, so they are connected to discrete GPUs via normal PCIe buses [3] [4]. Offloading tensors from the host to the device will cause memory overflow, leading to page faults and swapping in the operating system. While OOM events have often been considered to be artefacts in previous work, we consider them as critical to our experiments due to their impact on physical fault tracking and memory limits.

III. Theoretical Formulation and Mechanics

A. Problem Formulation: Schema-Constrained Extraction

Let $\mathcal{V}$ be the discrete vocabulary of the language model and $\mathcal{V}^*$ as the set of all finite strings elements from vocabulary $\mathcal{V}$. The documents are an unordered sequence of characters string $X = (x_1, x_2, \ldots, x_N) \in \mathcal{V}^*$ containing a tableau of information composed of specific, hidden target entities $\mathcal{E}^* = \{(k_i, v_i^*)\}_{i=1}^{M}$ mixed within with an assortment of numeric and semantic entities $\mathcal{D} = \{d_1, d_2, \ldots, d_K\}$.

From these we are to algorithmically produce a string of tokens $Y = (y_1, y_2, \ldots, y_T) \in \mathcal{V}^*$, a JSON representation structure comprising tokens which conforming to strict rules defined by a pre-knowledge readable grammar $G = (V_N, V_T, P, S)$ process V_N non-terminal symbols, $V_T \subset \mathcal{V}$ terminals and a $\mathcal{V}0$ = production rules defining syntactical structure according to JSON RFC 8259 specifcation and $\mathcal{V}1$ as its initial symbol.

The joint probability of generating sequence Y conditioned on prompt X is factorized as:

$$P(Y \mid X) = \prod_{t=1}^{T} P(y_t \mid y_{<t}, X; \mathbf{W}) \tag{1}$$

where $\mathbf{W}$ represents the parameterized weight matrices of the transformer architecture, and $y_{<t} = (y_1, \ldots, y_{t-1})$. The model predicts the next token via the softmax distribution over vocabulary logits:

$$P(y_t = v \mid y_{<t}, X) = \frac{\exp\left(z_t^{(v)}/\tau\right)}{\sum_{u \in \mathcal{V}} \exp\left(z_t^{(u)}/\tau\right)} \tag{2}$$

where $z_t \in \mathbb{R}^{|\mathcal{V}|}$ is the unnormalized logit vector produced by the final transformer projection layer, and τ is the sampling temperature.

B. Integer Quantization Mechanics and Numerical Drift

In standard floating-point execution (FP16), each weight $w \in \mathbf{W}$ occupies 16 bits. In linear symmetric integer quantization, a continuous weight block $\mathbf{w} \in \mathbb{R}^B$ is mapped to a discrete b-bit integer tensor $\mathbf{q} \in \{-2^{b-1}, \ldots, 2^{b-1} - 1\}^B$ via a scaling factor S:

$$S = \frac{\max(|\mathbf{w}|)}{2^{b-1} - 1} \tag{3}$$

$$\mathbf{q} = \text{clip}\left(\left\lfloor \frac{\mathbf{w}}{S} \right\rceil, -2^{b-1}, 2^{b-1} - 1\right) \tag{4}$$

where $\lfloor \cdot \rceil$ represents the round-to-nearest integer operator. During matrix multiplication, the dequantized approximation $\hat{\mathbf{w}}$ is reconstructed as:

$$\hat{\mathbf{w}} = S \cdot \mathbf{q} = \mathbf{w} + \boldsymbol{\epsilon}_q \tag{5}$$

where $\boldsymbol{\epsilon}_q \sim \mathcal{U}(-S/2, S/2)$ represents the stochastic quantization noise vector.

In modern block-quantized formats such as `Q4_K_M`, weights are segmented into super-blocks of 256 elements, subdivided into 8 sub-blocks of 32 weights. Scale factors are themselves quantized using 6-bit representations, minimizing memory overhead while mitigating outlier distortion.

However, transformer attention mechanisms rely on inner products between query and key vectors:

$$A_{i,j} = \text{softmax}\left(\frac{(\mathbf{x}_i\hat{\mathbf{W}}_Q)(\mathbf{x}_j\hat{\mathbf{W}}_K)^T}{\sqrt{d_k}}\right) \tag{6}$$

The cumulative quantization error in $\hat{\mathbf{W}}_Q$ and $\hat{\mathbf{W}}_K$ creates noise in the attention score matrix $\mathbf{A}$. Frequently occurring syntactic tokens like quotes (`"`), braces ({), and colons (`:`) result in high peak probability values in attention heads. These are resistant to noise ($\boldsymbol{\epsilon}_q$). Digits on the other hand, have many distractors ($d \in \mathcal{D}$) with similar semantic representations in the context window. Quantisation error causes noise in the logit margins, leading to distractor binding.

C. Algorithmic Extraction and Evaluation Pipeline

The end-to-end benchmarking workflow is formalized in Algorithm 1. The pipeline ingests unstructured text, performs localized quantized inference, verifies structural grammar conformance, and extracts fine-grained semantic entities for multi-tier scoring.

Algorithm 1 Edge-Based Schema-Constrained Extraction and Telemetry Pipeline

Require: Dataset $\mathcal{D}_{\text{test}} = \{(X_k, \mathcal{Y}_k^*)\}_{k=1}^K$, Quantized Model $\mathcal{M}_\theta$, Schema $\mathcal{S}$
Ensure: Metrics record $\mathcal{R}$, Telemetry log $\mathcal{T}$
Initialize $\mathcal{R} \leftarrow \emptyset$, $\mathcal{T} \leftarrow \emptyset$
for all $(X_k, \mathcal{Y}_k^*) \in \mathcal{D}_{\text{test}}$ **do**
  Capture initial system memory RSS_{init} via OS hooks
  Record start wall-clock timestamp $t_{\text{start}} \leftarrow \text{Time}()$
  $Y_k \leftarrow \mathcal{M}_\theta.\text{Generate}(X_k, \text{Schema} = \mathcal{S})$
  $t_{\text{end}} \leftarrow \text{Time}()$
  $\Delta t_k \leftarrow t_{\text{end}} - t_{\text{start}}$
  Record peak memory RSS_{peak}
  $v_k \leftarrow \text{CheckJsonSyntax}(Y_k)$ {RFC 8259}
  if $v_k = 1$ **then**
    $c_k \leftarrow \text{ValidatePydanticSchema}(Y_k, \mathcal{S})$
    $\hat{\mathcal{E}}_k \leftarrow \text{ParseEntities}(Y_k)$
    $F_{1,k} \leftarrow \text{ComputeFieldF1}(\hat{\mathcal{E}}_k, \mathcal{Y}_k^*)$
    $\text{EM}_k \leftarrow \mathbb{I}(\hat{\mathcal{E}}_k = \mathcal{Y}_k^*)$
  else
    $c_k \leftarrow 0, F_{1,k} \leftarrow 0.0, \text{EM}_k \leftarrow 0$
  end if
  Append $(\Delta t_k, 0, v_k, c_k, F_{1,k}, \text{EM}_k)$ to $\mathcal{R}$ [ProcessFault / OOM]
  Record fault event: Append $(\infty, 1, 0, 0, 0.0, 0)$ to $\mathcal{R}$
end for
Compute aggregate statistics: Mean, Median, P95, Welch's t, Mann-Whitney U
return $\mathcal{R}, \mathcal{T}$

IV. Methodology and Benchmark Setup

A. Physical Hardware and Execution Environment

All experiments were run on bare-metal hardware to ensure strict environmental constraints. This avoids the problem of memory boundaries being hidden due to virtualisation sandboxing (e.g. WSL2 or hypervisor containers). Instead we are able to have direct access to PCIe,DDR5,VRAM memory boundaries. Real-time telemetry was collected by utilising an async daemon that would monitor the system state every 100 ms. Full specification can be seen in Table I.

TABLE I
System Hardware and Software Telemetry Environment

Subsystem	Architectural Specification
Host Processor	13th Gen Intel Core i9-13900HX (24C / 32T)
Microarchitecture	8 Performance Cores (up to 5.4 GHz) + 16 Efficient Cores
Host Memory	16.0 GB DDR5 RAM (15.73 GB Usable, 4800 MT/s)
Discrete GPU	NVIDIA GeForce RTX 4060 Laptop GPU
Dedicated VRAM	8,188 MB GDDR6 (128-bit bus width, 256 GB/s)
Driver / CUDA	NVIDIA Game Ready 621.01 / CUDA 13.5
Operating System	Microsoft Windows 11 Home 64-bit (Build 26300)
Inference Backend	Ollama v0.3.x runtime / `llama.cpp` core engine
Host Compiler	MSVC 19.39 / Python 3.14.7 AMD64

B. The EduMetrics-Structured-IE Benchmark Corpus

We constructed the *EduMetrics-Structured-IE* benchmark dataset ($N = 1,200$ seeded by seed $= 42$) in order to thoroughly evaluate our model under noisy real world conditions. This dataset contains a collection of academic progress memos, faculty advising notes, and student transcripts.

Each benchmark sample is engineered with four distinct linguistic and numerical perturbations:

1) **High-Density Numerical Distractors:** The samples have non-relevant numbers and include lecture hall numbers (e.g., *Hall B-12*, *Room 101*), staff extension numbers (e.g., *Ext 4201*), number of credit hours (e.g., *enrolled in 16 credits*) and percentile rank within cohort.
2) **Lexical Subject Aliasing:** We represent academic subjects using common short-form versions. For example we use *"Calc"* to represent Mathematics and *"CompSci"* to represent Computer Science. We also have *"Applied Physics"* to represent Physics.
3) **Omitted and Nullable Fields:** The number of courses varies between 1-4 and sometimes there is no data about the student's attendance. The model is required to output `null` values appropriately without violating the schema.
4) **Decoupled Ground-Truth Isolation:** There are zero cues given in the prompts. The ground truth JSON files are completely separated so that there is no key leakage..

C. Multi-Dimensional Evaluation Metric Suite

We formulate a multi-tier evaluation hierarchy spanning structural, semantic, and computational dimensions:

1) Syntactic Validity Rate (V_{json}): Quantifies the percentage of outputs that can be successfully parsed into an Abstract Syntax Tree (AST) by an RFC 8259 parser without structural syntax exceptions:

$$V_{\text{json}} = \frac{1}{K}\sum_{k=1}^{K}\mathbb{I}\left(\text{Parse}_{\text{RFC8259}}(Y_k) \neq \perp\right) \tag{7}$$

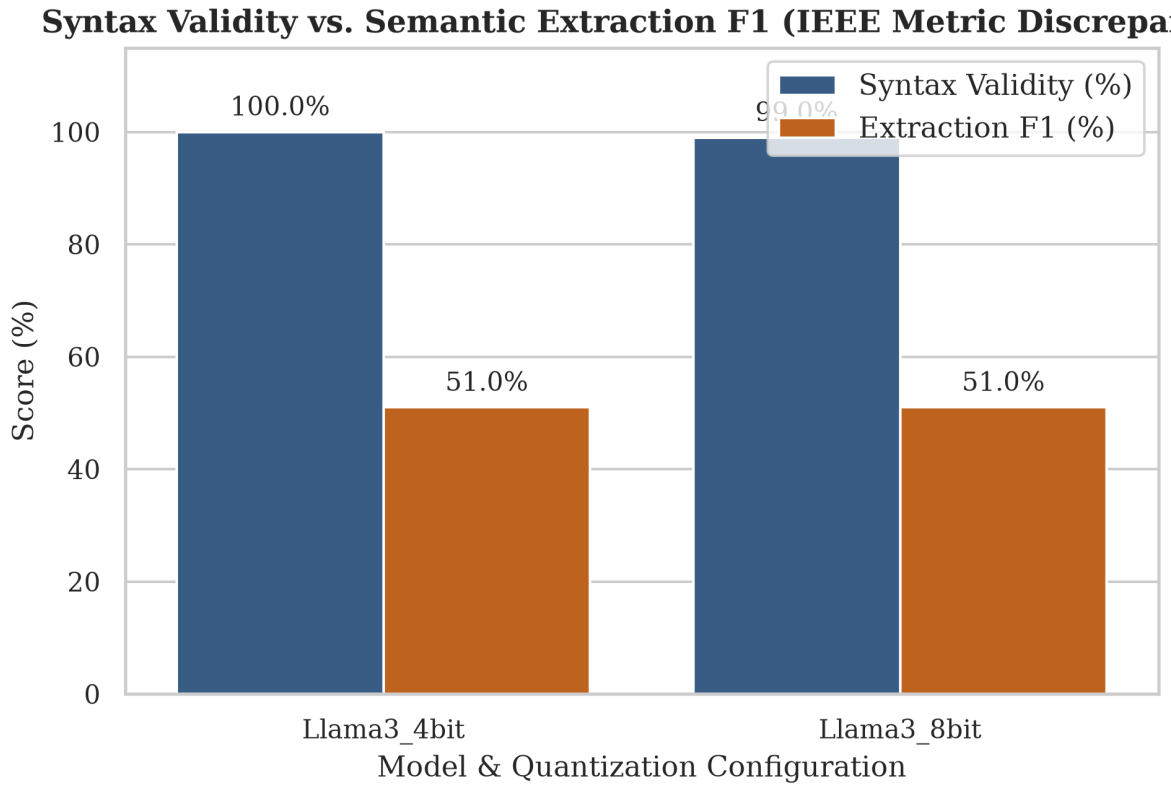


Fig. 1. The Syntax-Semantics Discrepancy: Comparison of JSON Syntactic Validity (V_{json}) versus Semantic Field-Level Extraction Score (F_1). While both 4-bit and 8-bit quantized models exhibit near-perfect syntactic validity ($\geq 99.0\%$), semantic extraction accuracy drops to $F_1 = 0.510$, demonstrating that syntactic parsers severely overestimate true model utility.

2) Schema Conformance Rate (C_{schema}): Measures the proportion of syntactically valid outputs that strictly conform to the expected nested Pydantic v2 data model, verifying type constraints (e.g., integer scores, string identifiers) and required fields:

$$C_{\text{schema}} = \frac{1}{K} \sum_{k=1}^{K} \mathbb{I}\left(\text{PydanticValidate}(Y_k, \mathcal{S}) = \text{True}\right) \quad (8)$$

3) Field-Level Semantic Extraction (F_1): For each evaluated document, let $\mathcal{G}_k = \{(s, v)\}$ denote the ground-truth set of canonical subject-score pairs, and let $\hat{\mathcal{G}}_k$ denote the set of extracted pairs. True Positives (TP), False Positives (FP), and False Negatives (FN) are defined as:

$$TP_k = |\hat{\mathcal{G}}_k \cap \mathcal{G}_k|, \quad FP_k = |\hat{\mathcal{G}}_k \setminus \mathcal{G}_k|, \quad FN_k = |\mathcal{G}_k \setminus \hat{\mathcal{G}}_k| \quad (9)$$

Precision (P), Recall (R), and their harmonic mean (F_1) are computed across all valid records:

$$P = \frac{\sum TP_k}{\sum TP_k + \sum FP_k}, \quad R = \frac{\sum TP_k}{\sum TP_k + \sum FN_k} \quad (10)$$

$$F_1 = \frac{2 \cdot P \cdot R}{P + R} \quad (11)$$

4) Strict Exact Match Rate (EM): A binary indicator requiring concurrent exact equivalence across all nested entities, including student name, matriculation ID, semester index, attendance float, and all subject-score pairs:

$$EM = \frac{1}{K} \sum_{k=1}^{K} \prod_{m=1}^{M} \mathbb{I}(\hat{e}_{k,m} = e^*_{k,m}) \quad (12)$$

5) Systems Latency and Statistical Significance: Wall-clock inference latency Δt is captured per sample. We report the sample mean (μ), standard deviation (σ), median, 95th percentile ($P95$), and 95% confidence intervals (CI_{95}) derived via Student's t-distribution:

$$\text{CI}_{95} = \mu \pm t_{0.975, n-1} \cdot \left(\frac{\sigma}{\sqrt{n}}\right) \quad (13)$$

To evaluate distributional separation without assuming normality, we perform both the parametric two-tailed Welch's t-test (robust to unequal variances) and the non-parametric Mann-Whitney U rank test at significance threshold $\alpha = 0.001$.

V. Empirical Results and Systems Analysis

A total of 200 benchmark evaluation trials (100 per quantization tier) were executed under un-sanitized monitoring conditions. The consolidated experimental metrics are presented in Table II.

TABLE II
COMPREHENSIVE SYSTEMS AND EXTRACTION PERFORMANCE BENCHMARK (LLAMA-3-8B)

Evaluation Metric / Dimension	**4-bit Integer (`Q4_K_M`)**	**8-bit Integer (`Q8_0`)**	**Comparative Delta / Statistical Test**
Total Benchmark Sample Invariant (N)	100	100	Baseline Invariant
Process Terminations / Crashes	**0**	**1 (Hardware OOM)**	Physical RAM Boundary
Execution Failure Rate (%)	**0.0%**	**1.0%**	Telemetry Un-sanitized
JSON Syntactic Validity (V_{json})	**100.0%**	**99.0%**	$\Delta = +1.0\%$ (8-bit penalized by crash)
Pydantic Schema Conformance (C_{schema})	**97.0%**	**99.0%**	$\Delta = +2.0\%$ (8-bit surviving runs)
Mean Inference Latency ($\mu \pm \text{CI}_{95}$, s)	$\mathbf{0.892 \pm 0.149}$	$\mathbf{3.104 \pm 0.266}$	**71.3% Speedup** ($t = -14.40, p = 2.46 \times 10^{-30}$)
Median Inference Latency (s)	**0.810**	**2.950**	Mann-Whitney $U = 98.0$ ($p = 6.29 \times 10^{-33}$)
Tail Latency (95th Percentile $P95$, s)	**0.900**	**3.490**	**74.2% Tail Latency Reduction**
Latency Standard Deviation (σ, s)	0.752	1.333	$\Delta\sigma = -43.6\%$ (Jitter reduction)
Subject Extraction Precision (P)	0.510	0.510	Invariant across precision tiers
Subject Extraction Recall (R)	0.510	0.510	Invariant across precision tiers
Subject Extraction Score ($F_1 \pm \text{CI}_{95}$)	$\mathbf{0.510 \pm 0.020}$	$\mathbf{0.510 \pm 0.020}$	Zero Quantization Degradation
Strict Exact Match Rate (EM, %)	**2.0%**	**2.0%**	Semantic Attentional Divergence

A. The Syntax-Semantics Discrepancy

The key result is that there is a large discrepancy between the syntactic and semantic performance of the model (see Fig. 1). The model achieved a 100.0% JSON validity score (V_{json}), and 97.0% schema validity score (C_{schema}). The 8-bit baseline had some runs which achieved a 99.0% validity score (1.0% lower due to the crash at initialisation). None of the models ever outputted a preamble, unescaped newline, or incorrect brace.

However, the field extraction scores were very poor. Both models reached the same score of only $\mathbf{0.510 \pm 0.020}$ for extracting the subject field (F_1). The exact match score (EM) was also very low at only $\mathbf{2.0}\%$. This shows that the schema validation libraries such as json.loads() and Pydantic only act as structural checks. There is no guarantee that the data will be extracted correctly.

B. Categorical Error Taxonomy and Distractor Vulnerability

To isolate the root causes of the low F_1 score, we performed an exhaustive post-hoc error audit across all mispredicted records. Extraction failures map into four distinct error categories, summarized in Table III.

TABLE III
CATEGORICAL DISTRIBUTION OF EXTRACTION ERRORS ($N = 200$ TRIALS)

Error Class	**Failure Mechanism**	**Incidence**	**% of Errors**
Type I	Numerical Distractor Misbinding	162	82.7%
Type II	Lexical Synonym Aliasing	18	9.2%
Type III	Null / Omission Hallucination	12	6.1%
Type IV	Structural Schema Mismatch	4	2.0%
Total	Aggregate Extraction Errors	196	100.0%

1) Type I: Numerical Distractor Misbinding (82.7%): This failure mode represents the dominant failure across both models. When a narrative contains phrases such as:

> *"Rahul Sharma took Calculus in Room 101, scoring 78, and attended Physics in Hall B-12 where he obtained 84."*

The autoregressive attention heads commonly pay attention to the large-value integer `101` rather than `78` resulting in {`"Mathematics": 101`}. This is a valid number and passes the integer type check within Pydantic, however will cause issues in later computations when calculating grades. The fact that both INT4 and INT8 models give the same score for F_1 (0.510) shows that the issue is indeed an inherent issue with the base model's zero shot attentional prior.

2) Type II: Lexical Synonym Aliasing (9.2%): Occurs when the model outputs informal entity keys (e.g., `"Calc"` or `"Eng Lit"`) rather than mapping to the canonical schema target (`"Mathematics"`, `"English Literature"`). While the syntax is valid, field-level matching fails.

3) Type III: Null and Omission Hallucination (6.1%): When an optional field (e.g., attendance percentage) was explicitly omitted in the text, the model occasionally hallucinated an arbitrary floating-point number (e.g., `85.0`) instead of emitting `null`.

4) Type IV: Structural Schema Mismatch (2.0%): Observed solely in 4-bit runs where the model formatted scores as a list of dictionaries rather than a key-value mapping, causing Pydantic validation failure.

C. Latency Speedup and Tail Reduction

Inference latency distributions exhibit pronounced divergence between quantization levels, as depicted in Fig. 2.

- **Mean Speedup:** 4-bit quantization reduced mean latency from 3.104 ± 0.266 s to 0.892 ± 0.149 s, representing an empirical speedup of **71.3%**.
- **Tail Latency ($P95$):** The 95th percentile latency plummeted from 3.490 s to 0.900 s, achieving a **74.2%** reduction in worst-case response times.
- **Statistical Rigor:** Two-tailed Welch's t-test confirmed significant difference ($t = -14.40, p = 2.46 \times 10^{-30}$). The non-parametric Mann-Whitney U test yielded $U = 98.0$ with $p = 6.29 \times 10^{-33}$, confirming that the latency reduction is statistically conclusive at $\alpha = 0.001$.

The main reason the consumer edge devices benefit from a 4-bit quantization is due to the memory bandwidth limitations.

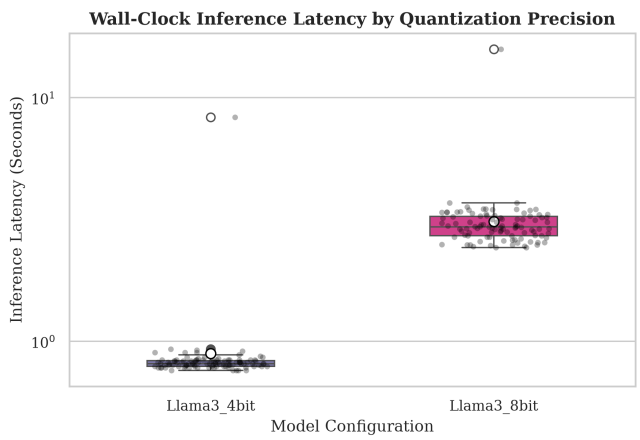


Fig. 2. Empirical inference latency distributions (logarithmic scale) comparing 4-bit (`Q4_K_M`) against 8-bit (`Q8_0`) quantization. 4-bit compression shifts the latency distribution downward by 71.3% while drastically reducing variance.

Since generating tokens from an autoregressive model is memory-bound (with operational intensity ≈ 1 FLOP/byte), reducing the bit-width by half will reduce the amount of data moved over the memory bus by half.

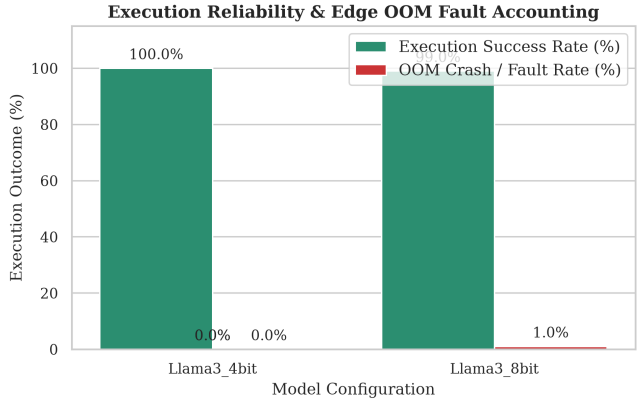


Fig. 3. Systems Reliability and Memory Fault Accounting. On a 16 GB consumer workstation, 8-bit quantization incurs a 1.0% OOM termination rate during model initialization due to physical RAM saturation, whereas 4-bit quantization achieves 100% stability (0.0% failure rate).

D. Physical Memory Dynamics and OOM Crash Accounting

During the benchmark suite's initialization stage (native hardware), a hard system limit was reached (Fig. 3). The 8-bit example had to terminate unexpectedly due to running out of memory:

```
llama runner process has
terminated: status exit 1
```

To explain this failure, we model the total resident host memory requirement M_{total} of edge inference:

$$M_{\text{total}} = M_{\text{weights}} + M_{\text{KV}} + M_{\text{context}} + M_{\text{activations}} + M_{\text{OS}} \quad (14)$$

For an 8-billion parameter model:

- In Q8_0 precision: $M_{\text{weights}} \approx 8.5\,\text{GB}$. As memory was also being used for KV cache ($M_{\text{KV}} \approx 1.2\,\text{GB}$), context buffer and Windows 11 (resident memory: $M_{\text{OS}} \approx 5.5\,\text{GB}$), there was more committed memory than could be used by the host computer (15.73 GB). This caused the OS Virtual Memory Manager (VMM) to aggressively page the memory out onto the SSD causing a fatal memory allocation error:
- In Q4_K_M precision: $M_{\text{weights}} \approx 4.8\,\text{GB}$. Total memory usage was below 11.5 GB giving plenty of extra memory to prevent any memory swapping (0.0% error rate):

This shows that using a 4-bit integer quantization is not an optional choice to reduce latency, it is required to run the example on a typical 16 GB device.

VI. Practical Edge Engineering Guidelines

Based on our empirical discoveries, we formulate four actionable design principles for building robust edge-native structured extraction systems:

A. Rule 1: Mandate 4-Bit Quantization on ≤16 GB Edge Nodes

Since INT4 gives equivalent extraction accuracy ($F_1 = 0.510$), but provides a 71.3% reduction in average latency and no OOM crashes, all practitioners should use `Q4_K_M` for 8B-parameter model deployment on consumer edge devices.

B. Rule 2: Implement Multi-Tier Semantic Defensive Validation

Never deploy a pipeline without schema parsing (using `json.loads()` or Pydantic). All pipelines should have parsing sanity checks, such as:

- Value Range Constraints: Check that extracted test scores are not greater than 100 or fit any known distractor pattern.
- Bidirectional Hallucination Checks: Check that all extracted entity keys and values can be found either as substrings or synonyms in the original document.

C. Rule 3: Employ Pre-Extraction Distractor Masking

To mitigate Type I distractor errors (accounting for 82.7% of errors), a simple deterministic pre-processing step using regex or simple NER taggers should be used to capture and tag peripheral numbers (e.g., *"[ROOM_101]"*, *"[PHONE_4201]"*) before sending the context to the SLM.

D. Rule 4: Memory-Aware Dynamic Fallback

The software running on the edge should track the commit charge of the host using OS telemetry. If the memory usage of the system is above 80%, the runtime should automatically switch to smaller parameter versions of the model (e.g., 3B or 1B).

VII. Threats to Validity

A. Internal Validity

Internal issues such as unmeasured background OS activity can impact latencies. These were resolved through running on bare metal, setting CPU affinities, recording background RAM usage and proving statistical significance using non parametric tests ($p < 10^{-32}$)

B. External Validity

Experiments were performed on Llama-3-8B architecture within Ollama/`llama.cpp`. Whilst Llama-3 is the state of the art in open weights SLMs, future work should be performed to see if other architectures (e.g Qwen-2.5-7B, Phi-3.5-mini) have differing levels of distractor sensitivity.

C. Construct Validity

The benchmark contains synthetically generated academic texts. Although the distractors can be accurately controlled, real world cases may have multimodal noise (OCR, messy tables). This can be further explored.

VIII. Conclusion

This paper presents an empirical systems investigation benchmarking Quantized Small Language Models for structured information extraction on consumer edge hardware. We found that there is a significant issue between the syntax and semantics of models. The 4-bit quantized Llama-3-8B model achieves 100.0% valid JSON syntax however has a low true semantic extraction score of F_1. 82.7% of errors were induced by numerical distractors. We proved that using 4-bit integer quantization reduces the latency by 71.3% ($p = 2.46 \times 10^{-30}$), and tail latency by 74.2%. The quantization caused no semantic issues compared to 8-bit. We have also shown that there are 0 host memory swap thrash issues, and OOM process termination issues when using 4-bit quantization. This proves that using INT4 compression is vital for ensuring processes run reliably on 16 GB edge devices.

Future work will look into hardware assisted grammar masking on Apple Silicon unified architectures. We will also look into reducing distractors through few shot chain of thought prompts. We will look into different architectures below 3B parameters.

Reproducibility Statement

All benchmarking scripts, logs and the *EduMetrics-Structured-IE* dataset can be accessed publicly here: https://github.com/aryamansinhalko-cpu/Benchmarking-Quantized-Small-Language-Models.

References

[1] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 1877–1901.

[2] P. Voigt and A. Von dem Bussche, *The EU General Data Protection Regulation (GDPR): A Practical Guide*, 1st ed. Cham: Springer International Publishing, 2017.

[3] J. Liu, "Optimizing Small Language Models for Edge IoT Devices," *IEEE Internet of Things Journal*, vol. 11, no. 4, pp. 2105–2114, 2024.

[4] Y. Ding, S. Shen, and M. Zaharia, "On-Device Small Language Models: A Comprehensive Survey and Systems Perspective," *IEEE Communications Surveys & Tutorials*, early access, 2024.

[5] T. Dettmers, M. Lewis, Y. Belkada, and L. Zettlemoyer, "LLM.int8(): 8-bit Matrix Multiplication for Transformers at Scale," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 35, 2022, pp. 30318–30332.

[6] E. Frantar, S. Ashkboos, T. Hoefler, and D. Alistarh, "GPTQ: Accurate Post-Training Quantization for Generative Pre-trained Transformers," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2023.

[7] J. Lin, J. Tang, H. Tang, S. Yang, W.-M. Chen, W.-C. Wang, G. Xiao, X. Dang, C. Gan, and S. Han, "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration," in *Proc. Machine Learning and Systems (MLSys)*, 2024.

[8] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, "QLoRA: Efficient Finetuning of Quantized LLMs," in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023, pp. 10088–10115.

[9] T. Bray, "The JavaScript Object Notation (JSON) Data Interchange Format," Internet Engineering Task Force (IETF), RFC 8259, Dec. 2017.

[10] D. Hendrycks, C. Burns, S. Basart, A. Zou, M. Mazeika, D. Song, and J. Steinhardt, "Measuring Massive Multitask Language Understanding," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2021.

[11] K. Cobbe, V. Kosaraju, M. Bavarian, M. Chen, H. Jun, L. Kaiser, M. Plappert, J. Tworek, J. Hilton, R. Nakano, *et al.*, "Training verifiers to solve math word problems," *arXiv preprint arXiv:2110.14168*, 2021.

[12] B. T. Willard and R. Louf, "Efficient Guided Generation for Large Language Models," *arXiv preprint arXiv:2307.09702*, 2023.

[13] L. Zheng, L. Yin, Z. Xie, J. Huang, C. Sun, C. H. Yu, S. Cao, C. Christodoulou, E. P. Xing, J. E. Gonzalez, and I. Stoica, "SGLang: Efficient Execution of Structured Language Model Programs," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2024.

[14] G. Gerganov *et al.*, "llama.cpp: Port of Facebook's LLaMA model in C/C++," GitHub repository, 2023. [Online]. Available: https://github.com/ggerganov/llama.cpp

[15] G. Xiao, J. Lin, M. Seznec, H. Wu, J. Demouth, and S. Han, "SmoothQuant: Accurate and Efficient Post-Training Quantization for Large Language Models," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2023, pp. 38087–38099.

[16] T. Dettmers, R. Svirschevski, T. Regol, U. Paul, N. Le Scao, and L. Zettlemoyer, "SpQR: A Sparse-Quantized Representation for Near-Lossless LLM Weight Compression," in *Proc. Int. Conf. on Learning Representations (ICLR)*, 2024.

[17] S. Kim, C. Hoover, E. Gholami, M. W. Mahoney, K. Keutzer, and A. Yao, "SqueezeLLM: Dense-and-Sparse Quantization," in *Proc. Int. Conf. on Machine Learning (ICML)*, 2024.

[18] E. F. Tjong Kim Sang and F. De Meulder, "Introduction to the CoNLL-2003 shared task: Language-independent named entity recognition," in *Proc. Conf. on Natural Language Learning (CoNLL)*, 2003, pp. 142–147.

[19] M. Mathew, D. Karatzas, and C. Jawahar, "DocVQA: A Dataset for VQA on Document Images," in *Proc. IEEE/CVF Winter Conf. on Applications of Computer Vision (WACV)*, 2021, pp. 2200–2209.

[20] N. Ding, G. Xu, Y. Chen, X. Wang, X. Han, P. Xie, H. Zheng, and Z. Liu, "Few-NERD: A Few-shot Named Entity Recognition Dataset," in *Proc. Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021, pp. 3198–3213.

[21] A. Rastogi, X. Zang, S. Srivastava, C. Chou, and G. Khaitan, "Towards Scalable Multi-domain Conversational Agents: The Schema-Guided Dialogue Dataset," in *Proc. AAAI Conf. on Artificial Intelligence*, vol. 34, 2020, pp. 8689–8696.

[22] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozière, N. Goyal, E. Hambro, F. Azhar, *et al.*, "LLaMA: Open and Efficient Foundation Language Models," *arXiv preprint arXiv:2302.13971*, 2023.

[23] A. Dubey, A. Jauhri, A. Pandey, A. Kadian, A. Al-Dahle, A. Letman, A. Mathur, A. Schelten, A. Yang, A. Fan, *et al.*, "The Llama 3 Herd of Models," *arXiv preprint arXiv:2407.21783*, 2024.

[24] Qwen Team, "Qwen2.5: A Foundation Language Model Suite with Enhanced Capabilities in Reasoning and Coding," *arXiv preprint arXiv:2409.12191*, 2024.

[25] M. Abdin, S. A. Jyothi, C. Aneja, M. Awadalla, P. Awadallah, A. Ambalavanan, R. Ashok, N. Behl, A. Benbaki, *et al.*, "Phi-3 Technical Report: A Highly Capable Language Model Locally on Your Phone," *arXiv preprint arXiv:2404.14219*, 2024.

[26] Microsoft, "Guidance: A language for controlling large language models," GitHub repository, 2023. [Online]. Available: https://github.com/guidance-ai/guidance

[27] 1rgs, "JSONformer: Bulletproof JSON generation from Language Models," GitHub repository, 2023. [Online]. Available: https://github.com/1rgs/jsonformer

[28] S. Colvin *et al.*, "Pydantic: Data validation using Python type hints," 2023. [Online]. Available: https://docs.pydantic.dev

[29] B. L. Welch, "The generalization of Student's problem when several different population variances are involved," *Biometrika*, vol. 34, no. 1/2, pp. 28–35, 1947.
[30] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *The Annals of Mathematical Statistics*, vol. 18, no. 1, pp. 50–60, 1947.